

Какой бэкенд учить фронтендеру в 2026?

Честный анализ рынка

1. Вступление — почему я вообще полез в это

Привет, Хабр! Меня зовут Сергей, я фронтенд-разработчик. 15 лет в профессии — кажется, целая вечность по меркам IT. React, TypeScript, финтех, фриланс, работа в небольшой, но гордой компании AWX Solutions — стандартный джентльменский набор человека, который видел ещё jQuery, ExtJS, Backbone, первые версии Angular и прочие артефакты древности.

Казалось бы, сиди и радуйся: стек востребован, зарплата капает, удалёнка позволяет работать из Ульяновска, не выходя из тапочек. Но где-то полгода назад я поймал себя на мысли, что упёрся в стеклянный потолок.

Не в зарплатный — с деньгами всё нормально. А в профессиональный. Понимаете, это чувство, когда ты пятнадцать лет делаешь «кнопочки и формочки» — пусть даже очень сложные, с анимациями на Framer Motion, с микрофронтендами на Module Federation, с хитрыми стейт-машинами на XState. Но рано или поздно наступает момент, когда хочется понимать, что там, под капотом. Как данные попадают в базу, почему API тормозит на ровном месте, и можно ли самому, вот этими самыми руками, поднять полноценный сервис от идеи до продакшена, не дёргая каждый раз бэкендера за рукав.

Fullstack, короче. Звучит солидно.

Я, как человек советской закалки (шучу, просто старый), не пошёл смотреть мотивационные ролики на YouTube с заголовками «Какой бэкенд учить в 2026? ТОП-5 ЯЗЫКОВ, ПОКА НЕ ПОЗДНО». Там обычно сидит парень с идеальной укладкой и рассказывает про язык, на котором он написал «Hello World» три месяца назад. Вместо этого я пошёл на hh.ru, заварил кофе покрепче и начал считать. Реальные вакансии. Реальные цифры. Без хайпа, без «мне кажется», без «мой знакомый сеньор сказал».

Спойлер: результаты меня удивили. И, возможно, удивят вас. Некоторые мифы подтвердились, некоторые — разбились вдребезги.

2. Как считал — методология (коротко, обещаю)

Сразу предупреждаю: я не data-сайентист, и мой анализ не тянет на диссертацию. Но для принятия карьерного решения точности хватает с запасом. Вот что я делал:

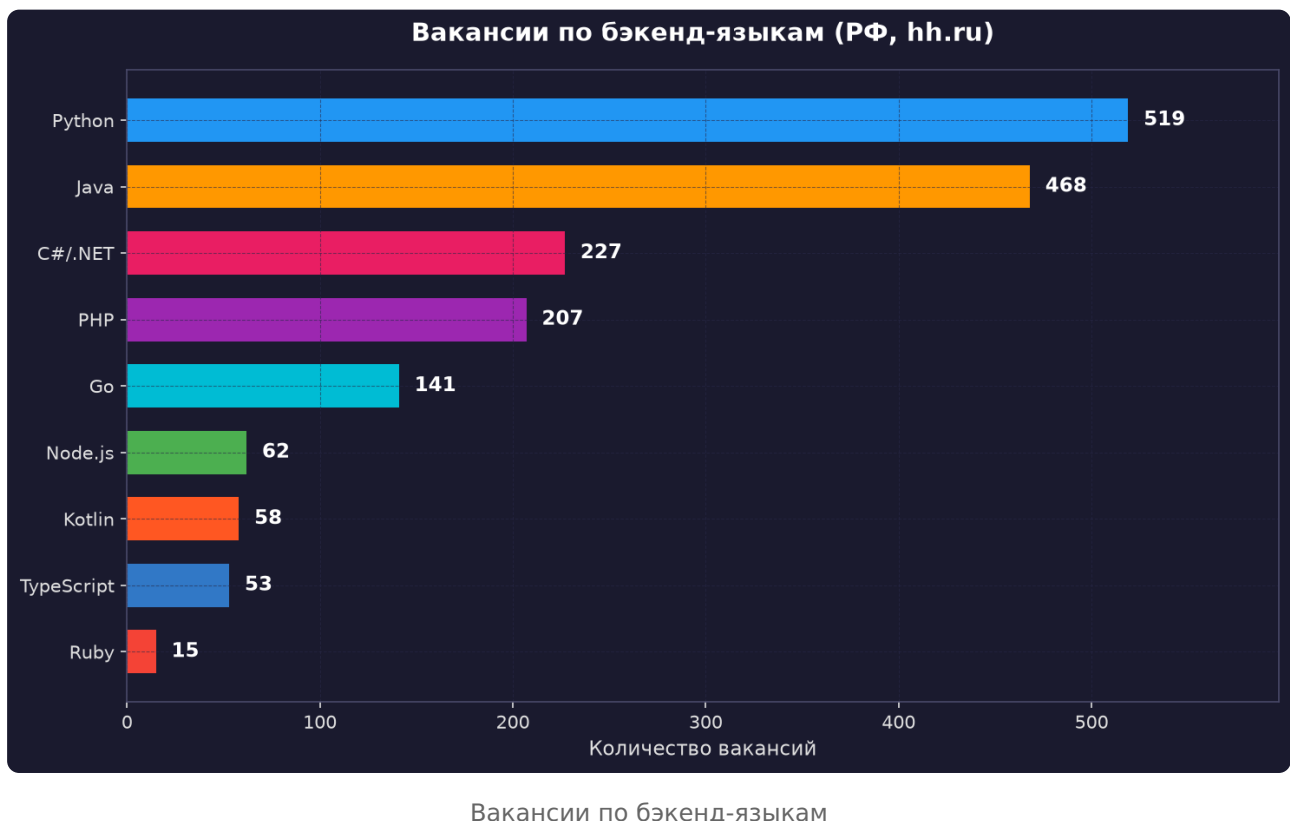
- **hh.ru, июль 2026, РФ.** Поиск строго по заголовкам вакансий. Никаких «искать в описании» — только прямые совпадения в названии должности. Сделал 15+ поисковых запросов с разными комбинациями: «Python разработчик», «Java разработчик», «fullstack разработчик», «fullstack Python», «React Node.js», «React PHP», «React Python» и так далее. Каждый запрос — с фильтром по России, без ограничения по зарплате и грейду.

- **Хабр Карьера** — для зарплатных данных. По состоянию на июль 2026 в выборке 43 716 анкет, так что цифры вполне репрезентативные. Медианные значения по грейдам, а не средние (чтобы зарплаты в Яндексе не искажали картину).
- **Ограничения метода.** Давайте честно: поиск по заголовкам не покрывает вообще все вакансии. Многие работодатели пишут «Senior разработчик» без указания конкретного языка в названии. Или «Fullstack Developer» без технологического стека в заголовке — а внутри ждут PHP/Laravel. Реальные цифры по каждому языку выше, чем в моей выборке. Но! Пропорции между языками этот метод отражает хорошо, потому что смещение работает примерно одинаково для всех. Грубо говоря, если Python-вакансий в заголовках в 3 раза больше, чем Go-вакансий, то и в реальности пропорция будет близкой.
- **Чего я не считал:** зарплаты в разрезе языков (это тема для отдельной статьи), продуктовые vs аутсорс-компании, стек внутри вакансии (там сотни комбинаций).

Если у вас есть знакомый HR, который даст доступ к расширенной аналитике hh.ru — маякните в комментариях, я дополню статью. За пиво, разумеется.

3. Рынок бэкенда в цифрах — общая картина

Первое, что я посчитал — общий рынок вакансий по языкам бэкенда. Не только fullstack, не только веб, а вообще все упоминания языка в заголовках. Вот что получилось:



- **Python** — 519 вакансий
- **Java** — 468
- **C#/.NET** — 227
- **PHP** — 207

- **Go** — 141
- **Node.js** — 62
- **Kotlin** — 58
- **TypeScript** — 53
- **Ruby** — 15

Первая реакция: Python — безоговорочный король, Java — королева, а остальные где-то там, в свите. Но давайте копнём чуть глубже и подумаем, что именно скрывается за этими цифрами.

Python — 519 вакансий. Выглядит как явный победитель. Но если открыть эти вакансии, значительная часть — это Data Science, Machine Learning, нейросети, аналитика данных, автоматизация тестирования. Не веб-разработка в классическом понимании. Python-разработчик в банке может вообще никогда не писать API — он обучает модели скоринга. Для fullstack-фронтендера это слабо релевантно, если только вы не планируете уходить в AI.

Java — 468 вакансий. Enterprise, банки, телеком, ритейл-гиганты. Высоконагруженные системы, микросервисы на Spring Boot. Это серьёзный, взрослый рынок. Но опять же — не всегда веб. Часть этих вакансий — чисто бэкенд, без фронта.

PHP — 207 вакансий. Тот самый язык, который «умер» ещё в 2012, 2015, 2018 и 2020 году, до сих пор имеет больше вакансий, чем Go и Node.js вместе взятые. Скажите спасибо Laravel, WordPress, Bitrix и миллиону интернет-магазинов на 1С-Битрикс.

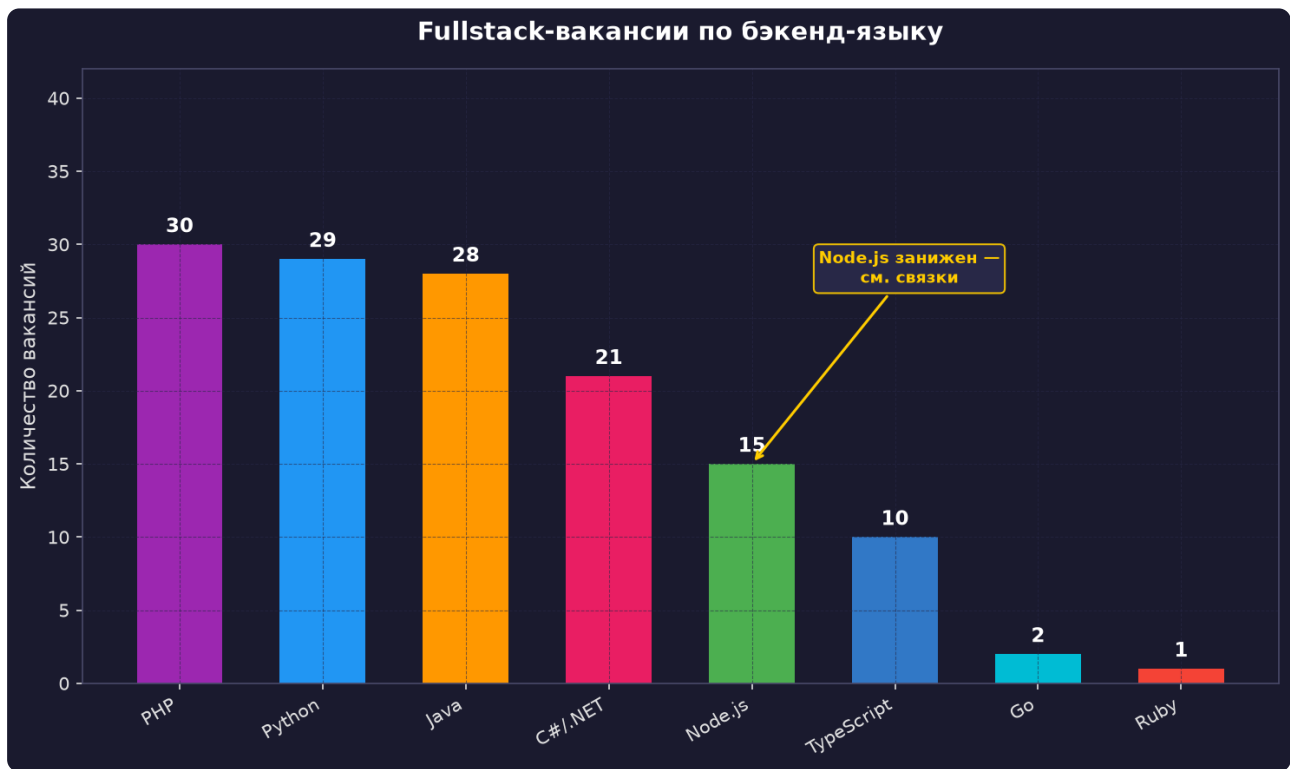
Node.js — 62, TypeScript — 53. А теперь важный момент: если TypeScript указан в заголовке как бэкенд-язык, это в 90% случаев NestJS или Next.js API routes. Технически — тот же Node.js. Если сложить Node.js и TypeScript — получаем 115 вакансий, уже третье место после Python и Java. Картина меняется, правда?

Go — 141. Внушительно для языка, которому нет и 15 лет. Но, как увидим дальше, это не про fullstack.

Ruby — 15. Когда-то был хайп, Rails-революция, стартапы. Сегодня — нишевый язык, в основном поддержка legacy.

4. Fullstack-срез: что ищут работодатели на самом деле

Когда я отфильтровал именно fullstack-вакансии (где в заголовке или первом абзаце описания явно указана связка фронтенд + бэкенд), картина радикально изменилась:



Fullstack-вакансии по бэкенд-языку

- **PHP** — 30 вакансий
- **Python** — 29
- **Java** — 28
- **C#/.NET** — 21
- **Node.js** — 15
- **TypeScript** — 10
- **Go** — 2
- **Ruby** — 1

Первый сюрприз: **PHP вырывается в лидеры**. Да-да, тот самый PHP, который «не язык, а поделка» и на котором «только WordPress пилят». В fullstack-нише у него первое место — 30 вакансий. Почему так?

Дело в том, что в экосистеме PHP исторически сложилась культура fullstack. Laravel — это фреймворк, который из коробки даёт и бэкенд, и шаблонизатор Blade, и часто разработчик один делает всё: от базы данных до вёрстки. Плюс огромное количество проектов — от интернет-магазинов до CRM-систем — где требуется «и швец, и жнец, и на дуде игрец». Я, честно говоря, был удивлён, но цифры есть цифры.

Python и Java — почти поровну, ~30 вакансий. Стабильный, предсказуемый спрос на Django/Flask/FastAPI и Spring Boot. Никаких сюрпризов.

Node.js — 15, TypeScript — 10. И вот здесь важный методологический момент. Если мы считаем TypeScript-бэкенд как разновидность Node.js (а это так и есть — NestJS, Next.js API routes, tRPC), то суммарно получаем 25 вакансий. Уже второе место, сразу за PHP.

Go — 2 fullstack-вакансии. И это не баг, а фича. Go не про fullstack. Go — это язык для высоконагруженных микросервисов, инфраструктурных инструментов, CLI-утилит,

Kubernetes-операторов. Когда компании нужен fullstack на Go — это экзотика. Скорее всего, какая-нибудь блокчейн-компания или стартап с нестандартным стеком.

Ruby — 1. Помянем. Rails был прекрасен, но рынок охладел. Скорее всего, эта одна вакансия — поддержка легаси-проекта, написанного в 2015 году.

Контекст рынка fullstack

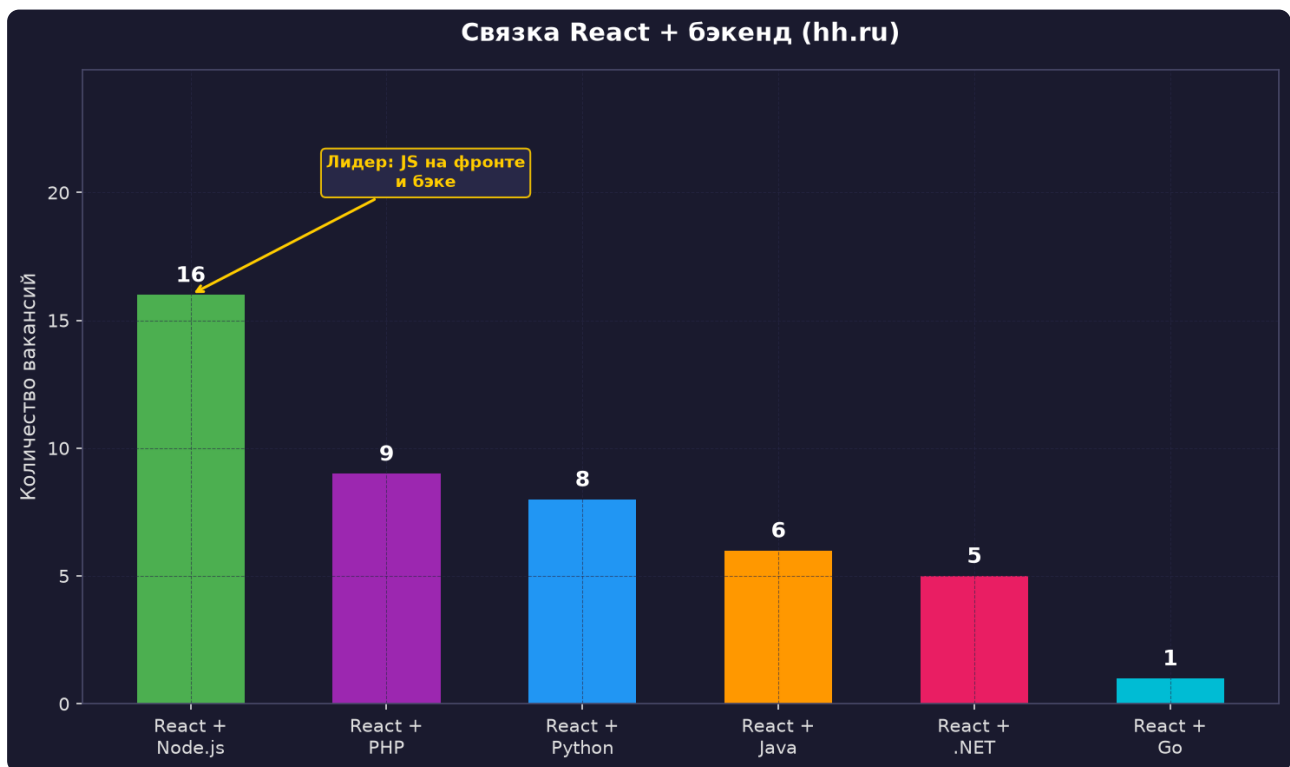
Несколько цифр, которые стоит держать в голове:

- Всего fullstack-вакансий в моей выборке: **330**
- Из них удалённых: **190 (58%)**
- Senior: **61**, Middle: **24**
- hh.индекс: **8.3** — это рынок работодателя. Конкуренция высокая.
- Прирост вакансий за последние месяцы: **+8%**, резюме: **−2%**

Рынок сложный, но тренд — в сторону соискателя. Медленно, но верно.

5. React + бэкенд: ключевой график для фронтендера

Теперь самое интересное — лично для меня и, подозреваю, для большинства читателей-фронтендеров. Я полез смотреть, какие бэкенды чаще всего встречаются в связке с React. Не «упомянут в скобках», а прямо в заголовке или первых строках описания:



React + бэкенд связки

- **React + Node.js** — 16 вакансий
- **React + PHP** — 9
- **React + Python** — 8
- **React + Java** — 6

- **React + .NET** — 5

- **React + Go** — 1

И вот здесь **Node.js показывает зубы по-настоящему**. В связке с React он абсолютный лидер с почти двукратным отрывом от ближайшего конкурента. 16 против 9 — это не случайная флуктуация, а системный паттерн.

Почему это логично и закономерно:

1. **Один язык на клиенте и сервере.** JavaScript/TypeScript везде — от кнопки «Отправить» до SQL-запроса. Меньше когнитивной нагрузки, не надо переключаться между синтаксисами. Типы можно переиспользовать: описал DTO один раз — и он работает и на фронте, и в API. Не надо синхронизировать две кодовые базы на разных языках.
2. **Next.js как точка сборки.** Next.js — фреймворк, который стирает границу между фронтом и бэком. API routes позволяют написать бэкэнд прямо в проекте на React. Server Components позволяют ходить в базу данных из компонента. Server Actions позволяют вызывать серверные функции как обычные async-функции на клиенте. По сути, любой Next.js-разработчик уже немножко fullstack.
3. **NestJS для «взрослых» проектов.** Если Next.js хорош для стартапов и 中小-проектов, то NestJS — это фреймворк для enterprise. Модульная архитектура, dependency injection, декораторы, guards, interceptors, pipes — всё как в нормальном бэкэнд-фреймворке, только на TypeScript.
4. **Единая экосистема пакетов.** Zod для валидации — работает на клиенте и сервере. date-fns — работает везде. shared-библиотеки в монорепе — без дублирования кода. Это не просто удобно, это даёт реальный буст к скорости разработки.

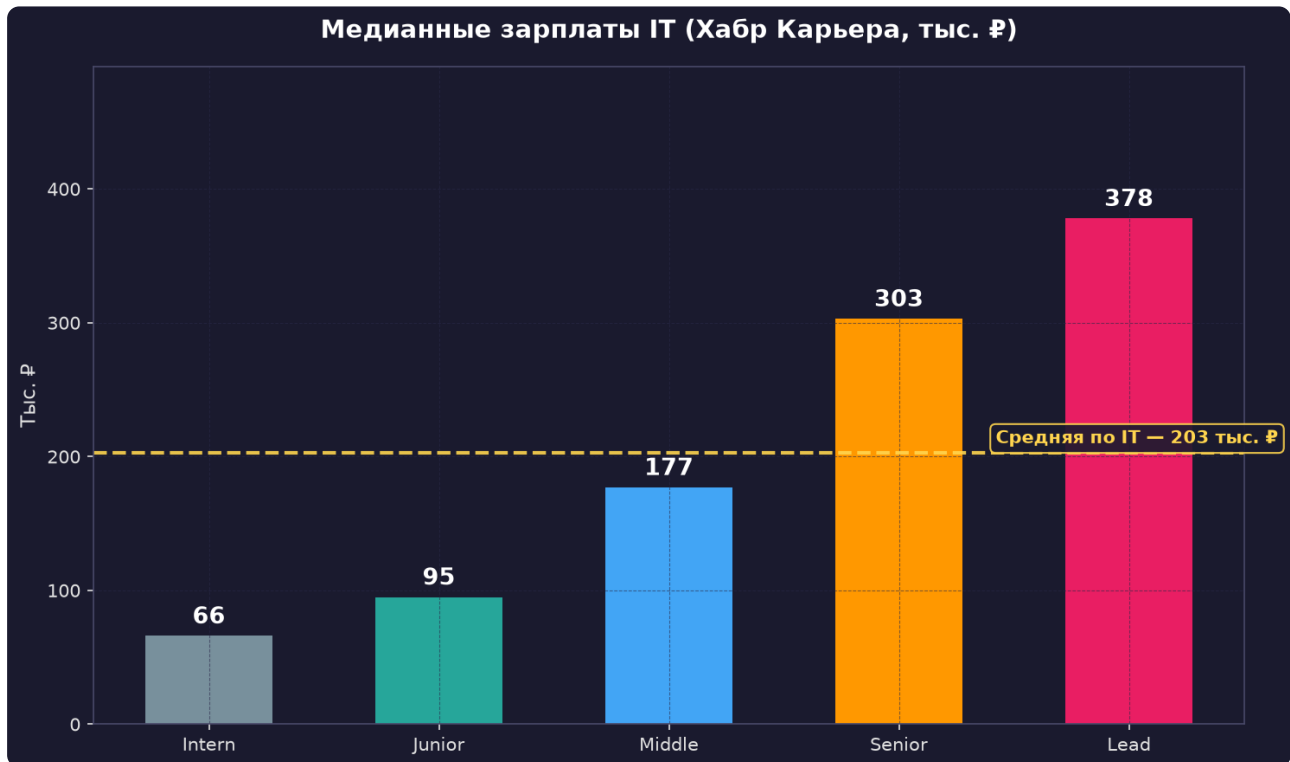
React + PHP (9 вакансий) — это, как правило, Laravel в бэкэнде и React в качестве админки или SPA-фронтенда. Рабочая связка, особенно в e-commerce и контентных проектах. Но будьте готовы к переключению между мирами JavaScript и PHP.

React + Python (8) — Django REST Framework или FastAPI на бэке, React на фронте. Хороший выбор, если проект завязан на Python-экосистему: AI, обработка данных, научные расчёты. Но таких проектов меньше, чем чисто веб-приложений.

React + Go (1) — экзотика. Если вы встретили эту вакансию, напишите в комментариях, очень интересно, что за проект.

6. Зарплаты и грейды — что с деньгами

Прежде чем бежать учить Node.js, давайте посмотрим на деньги. Данные с Хабр Карьеры, 43 716 анкет, июль 2026. Я взял медианные значения по fullstack-разработчикам (не средние, чтобы зарплаты в Яндексе и Тинькофф не исказили картину):



Зарплаты по грейдам

- **Intern** — 66 000 ₽
- **Junior** — 95 000 ₽
- **Middle** — 177 000 ₽
- **Senior** — 303 000 ₽
- **Lead** — 378 000 ₽

Для контекста: средняя зарплата по всем IT-специальностям — 203 124 ₽ (180 000 оклад + 23 124 премия).

На что я обратил внимание:

Разрыв между Middle и Senior — почти двукратный. 177k против 303k. Это о многом говорит. Рынок готов хорошо платить за глубину, а не за ширину. Fullstack-middle, который умеет «всё понемногу», стоит меньше, чем fullstack-senior, который глубоко разбирается и во фронте, и в бэке. И это логично: работодателю не нужен человек, который напишет посредственный код на двух языках. Ему нужен тот, кто уверенно владеет обоими.

Senior-вакансий в 2,5 раза больше, чем Middle. Смотрите сами: в моей выборке 330 fullstack-вакансий, из них 61 — senior и всего 24 — middle. Это перевернутая пирамида. Рынок fullstack смещён в сторону опытных специалистов. Junior-позиций практически нет (я нашёл буквально пару штук).

Что это значит на практике? Fullstack — не entry-level профессия. Сначала станьте сильным фронтендером (или бэкендером), наработайте экспертизу в одной области, а потом расширяйтесь. Я со своими 15 годами во фронте — как раз целевая аудитория таких вакансий.

7. Удалёнка и география — можно ли из Ульяновска

Отдельная радость для меня лично и, думаю, для многих читателей из регионов: **190 из 330 fullstack-вакансий — удалёнка**. Это 58%. Больше половины рынка не привязано к офису и московской прописке.

Для меня это принципиально. Я живу в Ульяновске. Здесь нет штаб-квартир Яндекса, Тинькофф или Сбера. Ближайшие офисы крупных IT-компаний — в Казани или Самаре, и то не факт, что там есть Fullstack-отдел. Но с удалёнкой это не проблема. Я могу работать на московскую или питерскую компанию, сидя в своей квартире с видом на Волгу. И, судя по цифрам, я такой не один.

Интересный момент: из 190 удалённых fullstack-вакансий значительная часть — это аутсорсинговые компании и продуктовые стартапы. Enterprise-сектор (банки, телеком) пока осторожнее относится к полной удалёнке, хотя тренд меняется. Если ваша цель — банк на Java, будьте готовы, что удалённых позиций будет меньше.

8. Что всё это значит — анализ

Теперь давайте переварим эти цифры в практические выводы. Что из всего этого следует для фронтендера, который, как и я, хочет стать fullstack?

Node.js — скрытый король fullstack для React-разработчика

Смотрите, какая интересная картина. На общем рынке Node.js скромно держится с 62 вакансиями. В fullstack-нише у него 15 позиций — третье-четвёртое место. Казалось бы, не впечатляет.

Но как только мы сужаем фокус до связок с React — Node.js вырывается на первое место с 16 вакансиями и двукратным отрывом. А если добавить TypeScript-бэкэнд (который технически тот же Node.js, просто с другой сигнатурой в заголовке) — получаем уже 26.

Мораль: Node.js — не самый массовый язык на бэкэнд-рынке вообще. Но для React-фронтендера он самый частый выбор работодателей. И это логично: зачем нанимать человека, который будет переключаться между JavaScript на фронте и Python на бэке, если можно взять того, кто пишет на одном языке и делает всё быстрее?

Python — прекрасен, но fullstack-ниша узкая

Я люблю Python. Правда. Это элегантный, выразительный язык с потрясающей экосистемой. 519 вакансий на общем рынке — это мощно. Но.

Значительная часть этих вакансий — не веб. Data Science, Machine Learning, AI, автоматизация, DevOps-скрипты, тестирование. В fullstack-нише Python делит второе-третье место с Java — ~30 вакансий. Это достойно, но не подавляюще.

Учить Python для fullstack имеет смысл, если вы планируете двигаться в сторону AI/ML-проектов. Тогда комбинация «React-фронтенд + Python-бэкэнд для ML-моделей» — это реально редкое и дорогое сочетание. Но если ваша цель — просто «делать веб-приложения целиком», Node.js даст больше возможностей.

Go — мощный, но не fullstack

141 вакансия на общем рынке — Go на подъёме, это видно невооружённым глазом. Каждый второй стартап пишет микросервисы на Go. Но fullstack-вакансий с Go — две штуки на всю страну.

Это не случайно. Идиоматика Go — писать небольшие, сфокусированные сервисы, которые делают что-то одно, но делают это хорошо. Это идеально для микросервисной архитектуры, но плохо стыкуется с идеей fullstack, где один человек делает всё.

Если ваша цель — стать крутым бэкэнд-разработчиком в highload-проектах, Go — отличный выбор. Но fullstack-фронтендером на Go вы, скорее всего, не станете просто потому, что таких вакансий почти нет.

PHP, Java, C# — стабильно, но другая вселенная

PHP — 30 fullstack-вакансий, лидер общего зачёта. Java — 28, C# — 21. Цифры говорят: эти языки востребованы во fullstack.

Но давайте честно: если вы 15 лет пишете на TypeScript с его строгой типизацией, дженериками, алгебраическими типами и всей этой красотой, переход на PHP — это культурный шок. Не потому что PHP плох (современный PHP 8.3 с типами и атрибутами — вполне приличный язык). А потому что экосистема, сообщество, подходы к архитектуре, фреймворки — всё другое.

Вы будете учить не просто синтаксис, а целый параллельный мир. С Node.js вы уже находитесь в этом мире — вы просто идёте чуть глубже.

9. Личный кейс: что выбрал я и почему

Ладно, хватит цифр. Давайте к личному.

После двух недель копания в hh.ru, десятков выписанных табличек, трёх литров кофе и пары бессонных ночей я принял решение. Барабанная дробь...

Мой выбор: Node.js + NestJS + TypeScript.

И вот почему — не потому, что «это модно», а потому что цифры сошлись с моим бэкграундом:

- 1. Один язык — одно сознание.** Я пишу на TypeScript больше пяти лет. Типы, дженерики, discriminated unions, условные типы — это мой рабочий инструмент каждый день. Когда я открываю NestJS-проект и вижу декораторы @Controller(), @Injectable(), @Get(), я не чувствую себя чужаком. Это просто продолжение моего TypeScript-опыта в другом контексте.
- 2. Лидер в React-связках — 16 вакансий.** Это не просто «Node.js популярен». Это конкретный сигнал от рынка: работодатели, которые ищут fullstack-разработчика на React, в первую очередь думают о Node.js на бэке. Мне не надо убеждать их, что я могу писать бэкэнд на TypeScript — они сами этого хотят.
- 3. Низкий порог входа — высокий потолок.** Базовая CRUD-APIшка на NestJS пишется за вечер: nest generate resource users, немного кода — и у вас работающий REST API с валидацией, Swagger-документацией и TypeORM. Но дальше — модульная

архитектура, CQRS, event-driven подходы, микросервисы на NATS/Kafka. Потолок высокий, расти есть куда.

4. **Next.js как трамплин.** Я уже работаю с Next.js на фронте. Server Actions, API Routes, middleware — это по сути и есть fullstack, просто на одном фреймворке. Когда я перехожу к NestJS, я не начинаю с нуля — я просто беру те же концепции и применяю их в более структурированном, enterprise-окружении.

5. **Typeshare.** Одно слово, а экономит недели разработки. DTO описываются один раз, и генерируются одновременно для NestJS (с декораторами class-validator) и для React-клиента (как чистые интерфейсы). Больше никаких «забыл обновить типы на фронте после изменения API».

Что дальше? После того как освою бэкенд на Node.js до уверенного senior-уровня, вторым языком в плане — Python с FastAPI. Не вместо Node.js, а в дополнение. Python открывает дверь в AI/ML-проекты, где я смогу использовать свой фронтенд-опыт для построения интерфейсов к ML-моделям. Это редкая и дорогая комбинация: человек, который понимает и как обучить модель, и как сделать удобный интерфейс для неё.

Вот мой текущий стек и резюме: <https://ulyanovsk.hh.ru/resume/e160b4d4ff10a11c320039ed1f645a73715876>

Если вы ищете fullstack-разработчика, который 15 лет оттачивал фронтенд (React, TypeScript, финтех, коммерческий опыт в AWX Solutions и на фрилансе как ИП) и теперь активно растёт в бэкенд на Node.js/NestJS — давайте обсудим. Я открыт к предложениям.

10. Рекомендации: какой бэкенд учить именно вам

Давайте соберём всё в практические рекомендации. Без абстрактного «зависит от ваших целей» — конкретно, с цифрами и аргументами:

Если ты React/TypeScript-фронтендер

Твой выбор — Node.js (NestJS / Next.js API routes).

Почему: самый быстрый вход (язык уже знаешь), лучшая синергия с текущим стеком, максимальное число вакансий в связке с React (16 штук, с отрывом от всех). Начни с NestJS — он даёт правильную архитектуру, модульность, dependency injection и enterprise-паттерны, которые пригодятся в любом языке. Бонус: если ты работал с Angular, NestJS покажется до боли знакомым — он буквально вдохновлён Angular.

Если ты data-сайентист или метишь в AI

Твой выбор — Python (FastAPI для нового, Django для «всего сразу»).

Python — безальтернативный лидер в AI/ML. Умение построить полноценное веб-приложение вокруг ML-модели (API для инференса, админка, визуализация результатов) делает тебя редким специалистом на стыке двух миров. FastAPI — для новых проектов с асинхронщиной и автодокументацией. Django — если нужен CRUD из коробки со встроенной админкой.

Если ты работаешь в enterprise или планируешь туда**Твой выбор — Java (Spring Boot) или C# (.NET).**

Банки, телеком, ритейл-гиганты, госсектор — здесь правят Java и C#. Это надёжный выбор с предсказуемым карьерным ростом и высоким чеком. Но будь готов: порог входа выше, экосистема тяжелее, а культура разработки отличается от JavaScript-мира. Зато стабильность и зарплаты на уровне.

Если ты любишь хардкор, производительность и инфраструктуру**Твой выбор — Go.**

Go — не про fullstack в классическом понимании. Это язык для высоконагруженных микросервисов, сетевых демонов, CLI-инструментов, Kubernetes-операторов. Если твоя мечта — писать системы, которые держат миллионы RPS и не потеют, Go создан для тебя. Но готовых fullstack-вакансий почти нет — придётся конструировать позицию самому или идти в чистый бэкенд.

Если тебе нужно максимальное число вакансий прямо сейчас**Твой выбор — PHP (Laravel).**

30 fullstack-вакансий — это больше, чем у любого другого языка. Laravel — зрелый, мощный фреймворк с отличной документацией и огромным комьюнити. Если твоя цель — найти работу как можно быстрее, PHP даёт больше всего вариантов. Но честно предупреждаю: переход из TypeScript-мира в PHP — это культурная адаптация. Готовься к динамической типизации, иной организации кода и другому сообществу.

11. Вывод

Рынок IT в России, июль 2026 — сложный, но понятный. hh.индекс 8.3 означает, что на одну вакансию приходится больше восьми резюме. Это рынок работодателя, и конкуренция высокая. Надо быть не просто хорошим, а заметным.

Но есть и обнадеживающие сигналы: количество вакансий растёт (+8%), а количество резюме падает (−2%). Медленно, но тренд разворачивается в сторону соискателя. Fullstack как специализация набирает обороты — и это шанс для тех, кто готов учиться.

Что я вынес из всего этого подсчёта — четыре главных урока:

1. Node.js — самый логичный выбор для React-фронтендера. Не потому что это хайп, не потому что «все побежали и я побежал». А потому что данные это подтверждают: 16 вакансий React + Node.js против 1 у React + Go. Разница в 16 раз — это не погрешность измерения, это сигнал рынка.

2. Fullstack — не для новичков. 61 senior-вакансия против 24 middle. Работодатели хотят глубину, а не ширину. Сначала станьте сильным фронтендером (или бэкендером), потом расширяйтесь. Со мной — 15 лет во фронте — именно так и получилось.

3. Один язык на клиенте и сервере — это конкурентное преимущество, а не просто удобство. TypeScript везде: типы можно шарить через монорепу, валидация одинаковая, инструменты общие. Это быстрее, надёжнее и, честно говоря, приятнее, чем держать в голове две экосистемы одновременно.

4. Данные важнее хайпа. Каждый месяц выходит статья «Почему язык X — будущее веб-разработки». Мог бы я пойти учить Rust, потому что «все говорят, что это новый C++»? Мог бы. Zig, Nim, Gleam — список можно продолжать. Но цифры с hh.ru говорят другое. И я предпочёл послушать цифры, а не восторженных блогеров.

Если вам понравился такой data-driven подход — ставьте плюс, добавляйте в закладки, пишите в комментариях, какой бэкенд выбрали вы и почему. Интересно сравнить реальный опыт людей с этими выкладками. Может быть, я что-то упустил, или у вас другая картина мира — давайте обсудим.

А если вы работодатель и вам нужен разработчик, который умеет не только код писать, но и принимать решения на основе данных — ссылка на резюме в разделе 9, буду рад пообщаться.

Дисклеймер: все данные собраны вручную с hh.ru и Хабр Карьеры в июле 2026 года, РФ. Возможны погрешности, связанные с методом поиска по заголовкам. Если вы видите неточность или у вас есть доступ к более полной аналитике — пишите в личку или комментарии, поправлю и дополню.